



ASHESI UNIVERSITY

SUPERMARKET DATABASE MANAGEMENT SYSTEM

Database Systems / Database Management

Charles Adjete

GROUP 3

Adama Baba - 79532028

Boniphace Benjamin - 75052028

Acheampong Panin Darkwah - 81152028

Adwoa Afriyie Adusei - 33442028

August 16, 2026

Table of Content

1. Introduction

1.1 Background

1.2 Problem statement

1.3 Motivation

1.4 Objective

1.5 Scope

2. Requirement Analysis

2.1 Stakeholders and users

2.2 Functional Requirement

2.3 Non-functional requirements

2.4 Assumptions

3. System Analysis

3.1 Existing system, Problems and proposed solution

3.2 Business rules

4. Database Design

4.1 ER diagram

4.2 Data dictionary

4.3 Relation and cardinality

5. Database Implementation

5.1 Relational schema

5.2 Normalization

5.3 Data dictionary (Tables, keys and constraints)

6. *SQL Implementation*

6.1 DDL and DML queries

6.2 Advanced queries.

6.3 Views

6.4 Procedures

6.5 Functions

6.6 Triggers

7. *Application design*

7.1 Architecture

7.2 User interface design

7.3 Modules

7.4 Navigation

8. *Application Implementation*

8.1 Description and screenshots of implemented functionality

9. *Testing*

9.1 Test strategy

9.2 Test cases

9.3 Expected results

9.4 Actual results

10. *Security and Data Integrity*

10. 1 Authentication

10. 2 Authorization

10.3 Constraints

10.4 Validation

10.5 Backup consideration

11. Challenges and Solution

12. Conclusion

13. Appendices

14. References

List of Tables

- 1. Relationships and cardinalities*
- 2. Category dictionary*
- 3. Customer dictionary*
- 4. LoyaltyCard dictionary*
- 5. Employee dictionary*
- 6. Discount dictionary*
- 7. Supplier dictionary*
- 8. Product dictionary*
- 9. Inventory dictionary*
- 10. ProductDiscount dictionary*
- 11. Purchase dictionary*
- 12. Sale dictionary*
- 13. PurchaseItem dictionary*
- 14. SaleItem dictionary*
- 15. InventoryMovement dictionary*
- 16. Supplier dictionary*

1. INTRODUCTION

1.1 Background: The retail sector is an important part of Ghana's economy. Small-scale supermarkets serving numerous people in their locality, providing them with access to beverages, household items, stationery, and other essential products. These small-scale supermarkets continue to rely on manual record-keeping methods, such as notebooks and spreadsheets,, making their operations very slow and difficult. This makes it difficult for them to maintain accurate and up-to-date records, track sales, manage customer information, and assess business performance. Inadequate record-keeping can also make it challenging for businesses to produce reliable financial information for decision-making and tax-related purposes (Yussif et al., 2019). Developing an integrated supermarket management system can help small-scale supermarkets improve performance and ensure good financial accountability.

1.2 Problem statement: Managing a supermarket is not just about keeping shelves stocked and cash registers running. It is a balancing act that requires effective inventory management, staff scheduling, customer relationship management, and financial record-keeping. However, most small-scale supermarkets in Ghana face significant challenges in managing these aspects of their daily operations.

Most supermarkets in Ghana face challenges with staff management, inventory tracking, customer data management, and customer retention strategies. These challenges often result in inventory inaccuracies, stock-outs, wastage of perishable goods, poor financial records, and reduced customer satisfaction. Over time, these inefficiencies contribute to financial losses and, in many cases, force supermarkets to shut down.

This is a growing concern because a majority of Ghanaian retail centres are dominated by traditional open-air markets, many of which lack structured financial management systems. As a result, they are unable to maintain accurate financial records that support informed business decisions and facilitate government tax collection. In a country where there is an increasing need to maximise tax revenue, providing small-scale supermarkets with effective management systems is essential to ensuring their sustainability while improving financial transparency and accountability. These people do not have structured financial statements that enable the

government to tax them. In a country where there is an increasing need to maximise tax collection, small-scale supermarkets should be provided with a better system to ensure their continued survival in the market.

1.3 Motivation: The motivation for developing this system comes from the need to provide small-scale supermarkets with a practical and affordable way to manage their business operations more efficiently. Manual record-keeping can be time-consuming and makes it easier for errors, missing information, and inconsistencies to occur. By addressing these issues, businesses can minimise loss, improve customer service, and support the long-term sustainability of small-scale supermarkets in Ghana.

1.4 Objective: Our objective as a team is to design an affordable, database-driven supermarket management system that provides an integrated solution for addressing the operational challenges faced by small-scale supermarkets in Ghana. The system aims to streamline daily operations, improve decision-making, enhance inventory and staff management, strengthen customer retention, and promote the long-term sustainability of these businesses despite the growing competition from larger retail stores and online vendors.

1.5 Scope: This project is designed to manage the supermarket business. It provides a quick way to operate by capturing manual processes and automating them. It is used to manage products, customers, suppliers, employees, and inventory; record sales transactions; process payments; and generate sales reports.

2.1 Stakeholders and users: The stakeholders are the individuals who are directly affected by the operations of our system or those who control and see to the functionality of the system. The system determines how their experiences at the supermarket are affected, whether it was able to process requests as quickly as possible, or whether it was able to attend to different customers simultaneously. The stakeholders include the supermarket owner, manager, staff, customers, suppliers, and database administrators. The users of our system are those who directly interact with it: employees, managers, and database administrators.

2.2 Functional Requirements

1. Product Management

- The system shall allow employees to add new products, delete discontinued products, and update product information, such as name, category, price, and expiry date.
- The system shall allow employees to search for products by name and show product availability and current stock levels.

2. Inventory Management: The system shall automatically update inventory levels after every sale, increase inventory when new stock is purchased from suppliers, notify employees when stock falls below a predefined reorder level, record inventory adjustments due to damaged, expired, or returned products, and generate inventory reports showing stock quantities and stock movements.

3. Supplier Management: The system shall allow employees to register new suppliers, store supplier information, including company name, contact person, address, phone number, and email, view products supplied by suppliers, update supplier information, and maintain purchase records associated with each supplier.

4. Purchase Management: The system shall allow employees to create purchase orders for suppliers, record the quantity and cost of purchased products, update inventory after purchased goods are received, maintain a history of all purchases, and generate purchase reports.

5. Sales Management: The system shall allow cashiers to process customer purchases, calculate the total cost of purchased items, apply taxes where applicable, generate a receipt after every completed sale, record the employee responsible for each sale, and maintain complete sales history.

6. Customer Management: The system shall register new customers, store customer information such as name, phone number, and email, update customer information, allow employees to search for customers, and maintain customers' purchase history.

7. Discount Management: The system shall allow employees to create promotional discounts, support both percentage-based and fixed-amount discounts, allow discounts to have start and end dates, automatically apply eligible discounts during checkout, and record discounts applied to each sale.

8. Loyalty Card Management: The system shall issue loyalty cards to registered customers, assign a unique loyalty card number to each customer, accumulate loyalty points after eligible

purchases, allow customers to redeem loyalty points during purchases, and display the customer's current loyalty points balance.

9. Employee Management: The system shall allow administrators to register employees, assign roles such as cashier, inventory officer, or manager, record which employee performs each transaction, and allow administrators to update employee information.

10. Reporting: The system shall generate daily, weekly, and monthly sales reports, generate inventory reports, generate supplier and customer purchase reports, and generate loyalty program reports.

2.4 Non-Functional Requirements

1. Performance: The system shall process a sales transaction within 3 seconds under normal operating conditions, return search results within 2 seconds, and support multiple users performing transactions simultaneously without significant delays.

2. Reliability: The system shall maintain data integrity even in the event of unexpected system failures, minimise transaction failures, and prevent duplicate records.

3. Availability: The system shall be available during supermarket operating hours, recover quickly after unexpected failures, and support regular database backups to minimise data loss.

4. Security: Sensitive customer information shall be protected from unauthorised access, and passwords shall be securely stored.

5. Usability: The system shall provide a simple and user-friendly interface that is consistent across all modules, and minimise the number of steps required to complete common tasks. Error messages shall clearly explain problems and suggest corrective actions.

6. Maintainability: The system shall be designed using modular components to simplify updates, follow normalisation principles to reduce redundancy, and support future enhancements such as online shopping or mobile applications.

7. Data Integrity: The system shall prevent duplicate product barcodes and ensure that sales cannot be recorded for products with insufficient stock.

8. Compatibility: The system shall be compatible with standard relational database management systems such as MySQL or MariaDB and support access from common web browsers if deployed as a web application.

Assumptions: The successful implementation of the Supermarket Management System assumes the following:

- Availability of necessary hardware infrastructure, including servers, computers, and mobile devices.
- Reliable internet connectivity for remote access to the system.
- Adequate training and support for supermarket personnel to adapt to the new system.
- Adherence to relevant regulations and compliance requirements related to data security, privacy, and financial transactions.
- The supermarket has a stable internet or local network connection.

3.1 System Analysis

The existing system in many small-scale supermarkets relies on manual record-keeping and spreadsheets, making it difficult to track various operations. To address this problem, our supermarket management system is designed to use centralised databases to store and manage information on products, customers, employees, suppliers, inventory, purchases, sales, and discounts. The system has a user-friendly interface that anyone can use without any database knowledge. The system will use appropriate tables, relationships, keys, constraints, stored procedures, functions, and triggers to improve data accuracy, automate important functions, and provide access to efficient information, such as daily reports, to support better decisions at the supermarket.

3.2 Business Rules

- Each product must have a unique ProductID
- Each customer must have a unique Customer ID
- Each sale must have a unique Sale ID
- Each supplier must have a unique Supplier ID
- Every supplier must supply one or more products

- Each product has one supplier
- Inventory must never be less than zero
- Every sale must have one or more products
- Exactly one employee must process each sale
- A customer can have many sales, but each sale is to only one customer
- Each loyalty card belongs to one customer
- Loyalty points are rewarded after successful purchases
- Products that are out of stock cannot be sold
- Discounts can only apply to valid products or active promotions
- Selling or receiving stock reduces and increases inventory quantity, respectively.

4.1 Entity Descriptions

Product: This entity describes products in the supermarket and includes attributes such as productID, productName, categoryID, supplierID, unitPrice, costPrice, expiryDate, barcode, and status.

Supplier: This is the entity that manages supplier information and the products they supply. It contains supplierID, supplierName, phoneNo, email, address, and city.

Purchase: It contains each purchase order placed by the supermarket with a supplier, with attributes including purchaseID, supplierID, employeeID, purchaseDate, totalCost, and status.

Sale: Represents each customer purchase made at the supermarket, with attributes including saleID, customerID, employeeID, saleDate, taxAmount, pointsRedeem, and paymentMethod.

Customer: Stores customer information of those who buy products and also have a loyalty membership. The attributes are: customerID, fName, lName, email, address, phone.

Employee: This entity stores details of supermarket staff and contains attributes such as employeeID, fName, lName, phone, email, hireDate, passwordHash, status, and position.

Inventory: This tracks the current stock level of every product in the supermarket. It contains attributes such as inventoryID, productID, quantity, reorderLevel, and lastUpdated.

PurchaseItem: This keeps the individual products included in each purchase transaction. Their attributes are: purchaseID, supplierID, productID, unitCost, and quantity. The combination of purchaseID and productID forms the composite primary key.

SaleItem: Stores the products sold during a particular sale. The attributes they contain are: saleID, productID, quantity, unitPrice, discount, and subTotal.

Discount: This entity stores promotional discount information that can be applied to products during specified periods. Its attributes are: discountID, discountName, discountPercent, fixedAmount, discountType, startDate, and endDate.

LoyaltyCard: Stores loyalty program information and reward points for customers. The attributes are: cardID, customerID, points, issueDate, expiryDate, and status.

Category: This entity organizes products into categories to make product classification and management. It contains attributes such as categoryID, categoryName, and description.

ProductDiscount: This entity represents the relationship between products and discounts, allowing discounts to be associated with specific products. It contains productID and discountID, which together form the composite primary key and are also foreign keys referencing Product and Discount, respectively.

InventoryMovement: This entity records changes made to the quantity of products in inventory, providing a history of stock movements. It contains attributes such as movementID, productID, employeeID, movementType, quantity, movementDate, and referenceID.

4.2 Relationships and cardinalities.

Entity 1	Relationship	Entity 2	Cardinality	Entity 1 Min/Max	Entity 2 Min/Max
Customer	owns	LoyaltyCard	1: 1	(0, 1)	(1, 1)
Customer	completes	Sale	1: N	(0, N)	(0, 1)
Employee	processes	Sale	1: N	(0, N)	(1, 1)
Sale	contains	Product	M: N*	(1, N)	(0, N)
Supplier	supplies	Product	1: N	(0, N)	(1, 1)

Category	classifies	Product	1 : N	(0, N)	(1, 1)
Purchase	contains	Product	M: N*	(1, N)	(0, N)
Supplier	receives a purchase from	Purchase	1: N	(0, N)	(1, 1)
Employee	records	Purchase	1: N	(0, N)	(1, 1)
Discount	applies to	Product	M: N*	(0, N)	(0, N)
Product	has	Inventory	1: 1	(1: 1)	(1: 1)
Employee	records	InventoryMovement	1: N	(0: N)	(1, 1)
Product	has	InventoryMovement	1: N	(0, N)	(1, 1)

*M: N relationships are represented directly in the conceptual Chen ERD. During relational implementation, these are resolved using the associative entities SaleItem, PurchaseItem and ProductDiscount.

5. 1 Relational Schema:

Customer(customerID PK, fName, lName, email, address, phone)

LoyaltyCard(cardID PK, customerID FK, points, issueDate, expiryDate, status)

Employee(employeeID PK, fName, lName, position, phone, email, hireDate, passwordHash, status)

Supplier(supplierID PK, supplierName, phoneNo, email, address, city)

Product(productID PK, supplierID FK, productName, categoryID FK, unitPrice, costPrice, expiryDate, status, barcode)

Category(categoryID PK, categoryName, description)

Inventory(inventoryID PK, productID FK, quantity, lastUpdated, reorderLevel)

Discount(discountID PK, discountName, discountPercent, fixedAmount, discountType, startDate, endDate)

ProductDiscount(productID PK, FK, discountID PK, FK)

Purchase(purchaseID PK, supplierID FK, employeeID FK, purchaseDate, totalCost, status)

PurchaseItem(purchaseID PK, FK, productID PK, FK, quantity, unitCost)

Sale(saleID PK, customerID FK, employeeID FK, saleDate, totalAmount, paymentMethod, taxAmount, pointsRedeemed)

SaleItem(saleID PK, FK, productID PK, FK, quantity, unitPrice, discount, subTotal(derived))

InventoryMovement(movementID PK, productID FK, employeeID FK, movementType, quantity, movementDate, referenceID)

5.2 Normalisation: The supermarket management database is designed to satisfy the principle of normalization up to the third Normal Form (3NF) to reduce data redundancy and prevent insertion, update and deletion anomalies (Connolly & Begg, 2015).

1st Normal Form (1NF): The database satisfies 1NF because each table contains atomic values, with no repeating groups or multi-valued attributes. Each record can be uniquely identified using a primary key. For example, customer details are stored as individual attributes in the Customer entity rather than as a combined field.

2nd Normal Form (2NF): The second normal form is satisfied because it is in 1NF and non-key attributes depend on the whole primary key. This is particularly important for entities such as PurchaseItem and SaleItem with composite primary keys. Attributes such as quantity and unitCost depend on the combination of the purchase and the product rather than only one part of the composite key. Likewise, the SaleItem attributes describe a specific product within a sale.

3rd Normal Form (3NF): The database satisfies 3NF because it is in 2NF, and non-key attributes do not depend on other non-key attributes. For example, supplier and category information is stored in the Supplier and Category relation, respectively, rather than being stored repeatedly in Product. This prevents supplier and category information from being unnecessarily duplicated across multiple product records.

In conclusion, the normalization process helped us reduce redundancies and prevent the anomalies(insertion,update & deletion) while maintaining the relationships required by the supermarket management system.

5. 3 Data Dictionary

Category

Attribute	Data Type	Size	Constraint	Description
categoryID	VARCHAR	10	PK	Identifier of categories
categoryName	VARCHAR	100	NOT NULL UNIQUE	Name of a particular category
description	VARCHAR	255		What the category is about, like the type of items that are in that category

Customer

Attribute	Data Type	Size	Constraint	Description
customerID	VARCHAR	10	PK	Unique identifier for each customer
fName	VARCHAR	50	NOT NULL	Customer's first name
lName	VARCHAR	50	NOT NULL	Customer's last name
email	VARCHAR	100	UNIQUE	Customer's email address
address	VARCHAR	255		Customer's residential address
Phone	VARCHAR	20	NOT NULL UNIQUE	Customer's contact number

LoyaltyCard

Attribute	Data Type	Size	Constraint	Description
cardID	VARCHAR	10	PK	Unique identifier for each loyalty card
customerID	VARCHAR	10	NOT NULL, UNIQUE, FK	Customer's ID
points	INT		DEFAULT 0	Points accumulated
issueDate	DATE		NOT NULL	Card's issue date
expiryDate	DATE		NOT NULL	Card expiry date
Status	ENUM		DEFAULT 'Active'	Status of card

Employee

Attribute	Data Type	Size	Constraint	Description
employeeID	VARCHAR	10	PK	Unique identifier for each employee
fName	VARCHAR	50	NOT NULL	Employee's first name
lName	VARCHAR	50	NOT NULL	Employee's last name
position	VARCHAR	50	NOT NULL	Employees role
phone	VARCHAR	20	NOT NULL, UNIQUE	Employee contact number
email	VARCHAR	100	UNIQUE	Employee email address
hireDate	DATE		NOT NULL	Date employee joined the organization
passwordHash	VARCHAR	255	NOT NULL	Hashed employee login password
status	ENUM		DEFAULT 'Active'	Employment status (Active/Inactive)

Discount

Attribute	Data Type	Size	Constraint	Description
discountID	VARCHAR	10	PK	Identifier of discounts available
discountName	VARCHAR	100	NOT NULL	Label of the discount
discountPercent	DECIMAL	(5, 2)	NOT NULL	Percentage of the discount
startDate	DATE		NOT NULL	When the discount starts to apply
endDate	DATE		NOT NULL	When the discount ends
discountType	ENUM		NOT NULL, DEFAULT 'Percentage'	The type of discount(percentage or fixed)
fixedAmount	DECIMAL	(10, 2)	DEFAULT 0	Discount value

Supplier

Attribute	Data Type	Size	Constraint	Description
SupplierID	VARCHAR	10	PK	Unique identifier for each supplier
supplierName	VARCHAR	100	NOT NULL	Name of supplier company/person
phoneNo	VARCHAR	20	NOT NULL UNIQUE	Supplier contact number
email	VARCHAR	100	UNIQUE	Supplier's email address
address	VARCHAR	225	NOT NULL	suppliers' house address
city	VARCHAR	100	NOT NULL	Place of residence

Product

Attribute	Data Type	Size	Constraint	Description
productID	VARCHAR	10	PK	Unique identifier for each product
supplierID	VARCHAR	10	NOT NULL, FK	Supplier is responsible for supplying the product
productName	VARCHAR	100	NOT NULL UNIQUE	Name of product
categoryID	VARCHAR	10	NOT NULL, FK	Product category (Food, Beverage, etc.)
unitPrice	DECIMAL	(10, 2)	NOT NULL, CHECK >= 0	Selling price per unit
Cost price	DECIMAL	(10, 2)	NOT NULL, CHECK >= 0	Purchase cost per unit
expiryDate	DATE			Product expiration date
status	ENUM		DEFAULT 'Availability'	Product availability status
barcode	VARCHAR	50	UNIQUE	Products identification barcode

Inventory

Attribute	Data Type	Size	Constraint	description
InventoryID	VARCHAR	10	PK	Unique inventory record identifier
productID	VARCHAR	10	NOT NULL, UNIQUE, FK	Product being tracked
quantity	INT		DEFAULT = 0, CHECK >= 0	Available quantity in stock
reorderLevel	INT		DEFAULT 10, CHECK >= 0	
lastUpdated	DATETIME		DEFAULT=CURRE	Date the inventory

			NT_TIMESTAMP	record was updated
--	--	--	--------------	--------------------

ProductDiscount

Attribute	Data Type	Size	Constraint	description
productID	VARCHAR	10	PK, FK	Product receiving THE discount
discountID	VARCHAR	10	PK, FK	Discount applied to the product

Purchase

Attribute	Data type	Size	Constraints	Description
purchaseID	VARCHAR	10	PK	Unique identifier for a purchase transaction
supplierID	VARCHAR	10	NOT NULL, FK	Supplier providing goods
employeeID	VARCHAR	10	NOT NULL, FK	The employee who recorded the purchase
purchaseDate	DATE		NOT NULL	The date the purchase was made
totalCost	DECIMAL	(12, 2)	DEFAULT 0, CHECK >= 0	Total cost of purchase
status	ENUM		DEFAULT 'Pending'	Purchase status

Sale

Attribute	Data Type	Size	Constraint	Description
saleID	VARCHAR	10	PK	Identifier of sales
customerID	VARCHAR	10	FK	Identifier of a customer
employeeID	VARCHAR	10	NOT NULL, FK	Employee identifier

saleDate	DATETIME		DEFAULT CURRENT_TIMESTAMP	The day the sale is made
paymentMethod	ENUM		NOT NULL	How the customer paid(through Momo, cash)
pointsRedeemed	INT		NOT NULL, DEFAULT 0, CHECK >= 0	Points won or accumulated
totalAmount	DECIMAL	(12, 2)	DEFAULT = 0, CHECK >= 0	Total amount of the sale
taxAmount	DECIMAL	(12, 2)	DEFAULT = 0, CHECK >=0	Tax applied to the sale

PurchaseItem

Attribute	Data Type	Size	Constraint	Description
purchaseID	VARCHAR	10	PK, FK	Identifier of purchase
productID	VARCHAR	10	PK, FK	Identifier of the product being purchased
quantity	INT		NOT NULL, CHECK > 0	The quantity of the product that is purchased
unitCost	DECIMAL	(10, 2)	NOT NULL, CHECK >= 0	The cost of each purchase of a product cost

SaleItem

Attribute	Data Type	Size	Constraint	Description
saleID	VARCHAR	10	PK, FK	Identifier of sale
productID	VARCHAR	10	PK, FK	Identifier of the

				product being sold
quantity	INT		NOT NULL, CHECK > 0	Quantity of product sold
unitPrice	DECIMAL	(10, 2)	NOT NULL, CHECK >= 0	How much the product is sold for
discount	DECIMAL	(5, 2)	DEFAULT = 0, CHECK >= 0 AND <= 100	Amount of discount applied to the product
subTotal	DECIMAL	(12, 2)	GENERATED	Automatically calculated subTotal after applying the discount

InventoryMovement

Attribute	Data Type	Size	Constraint	Description
movementID	INT(AUTO_INCREMENT)		PK	A unique identifier for each stock change.
productID	VARCHAR	10	NOT NULL,FK	Identifies the product whose inventory quantity changed
employeeID	VARCHAR	10	NOT NULL, FK	Identifies the employee who was responsible for the inventory movement.
movementType	ENUM		NOT NULL	Indicates why or how the inventory changed (purchase, return, damage)
referenceID	VARCHAR	10		Identifies the transaction that caused the movement. For example, if the movement resulted from a purchase, it can contain the

				corresponding purchaseId.
quantity	INT		NOT NULL, CHECK > 0	The number of units affected by the movement.
movementDate	DATETIME		NOT NULL, DEFAULT CURRENT_ TIMESTAMP	The date and time when the inventory movement occurred.

6. SQL Implementation

6.1 Views:

1. **ProductStockStatus:** Shows all products with their current stock level and a label (In Stock, Low Stock, or Out of Stock) based on the reorder level.
2. **CustomerPurchaseHistory:** Displays every item a customer bought across all their sales, including dates, prices, and applied discounts.
3. **ActiveDiscounts:** Lists only discounts that are running today, showing which products are on sale and their discount amounts.
4. **SupplierPurchaseSummary:** Summarizes how many times you've purchased from each supplier and the total amount spent with them.
5. **SalesSummary:** Groups sales by date and shows the total number of sales, revenue, and loyalty points redeemed per day.
6. **ProductsPerCategory:** Counts how many products are in each product category.

6.2 Procedures:

1. **ProcessSale:** Creates a new sale record and adds an item to it. Automatically fetches the product price and current discount, then inserts both the sale and the sale item into the database.
2. **SearchProducts:** Finds products by searching their name, barcode, or category. Returns product details, including ID, name, price, stock quantity, and status in one query.

3. **RecordPurchase:** Records when you buy stock from a supplier. Inserts the purchase order and the items purchased, then calculates and updates the total cost of that purchase.

6.3 Functions:

1. **CalculateSaleTotal:** Takes a sale ID and adds up all the subtotals from that sale's items. Returns the total amount for that sale.
2. **GetProductDiscount : ProcessSale: Creates a new record. Looks up** what discount percentage applies to a product based on active discount dates. Converts fixed discounts to percentages and returns the highest applicable discount (capped at 100%).

6.4 Triggers:

1. **BeforeSaleItemInsert:** Prevents a sale if the product has no inventory or if you're trying to sell more than is in stock.
2. **AfterSaleItemInsert:** Automatically reduces inventory when an item is sold and logs the movement in the audit trail.
3. **AfterPurchaseItemInsert:** Automatically increases inventory when you receive stock from a supplier and records the purchase movement.

7. Application Design

7.1 Architecture: The Supermarket Management System was designed as a web-based application to help supermarket staff manage the main activities of the business efficiently. The system has a 3-tier architecture: the user interface, the application layer, and the database layer. The system uses a relational database to store and organise the information. The application communicates with the database via the backend, allowing information entered through the website to be stored, retrieved, updated, and deleted as needed.

7.2 User interface design: The user interface provides a friendly framework that users can easily navigate between sections. The system uses a dashboard as the main interface, where users can access the different management modules. The user interface comprises components that enable users to access the system's core functions.

- Login interface: The system allows users to enter their username, supermarket role, and password. This ensures that only registered people have access to the supermarket information.
- Dashboard interface: Provides an overview of the supermarket system and access to the main functions and different modules.
- Product Management interface: Allows users to add, view, edit, search, and delete product information.
- Sales Interface: This interface allows employees to select products, enter quantities, calculate the total, apply discounts where applicable, and complete a sale.

7.2 Modules

- Inventory Management: Tracks available stock and helps monitor inventory levels.
- Category Management: Allows products to be organised into different categories.
- Supplier Management: Stores and manages supplier information.
- Customer Management: Stores customer information and allows customer records to be managed.
- Employee Management: Manages information about employees who use or operate the system.
- Purchase Management: Records purchases made from suppliers and updates inventory.
- Sales Management: Records customer sales and automatically updates product stock.
- Discount Management: Allows discounts to be created and applied to eligible products.
- Loyalty Card Management: Enables recording and management of customer loyalty information.

7.3 Navigation

The system uses a simple navigation structure that allows users to access the different modules from a dashboard. After successfully logging in, the user is directed to the dashboard, where they can select a module and perform the appropriate tasks. Each module will provide options to add, view, edit, search for, or manage relevant information, depending on the user's role and level of accessibility.

8. Application Implementation

The Supermarket Management System was implemented as a web application using Python and Flask for the backend (Pallets Projects) , HTML and CSS for the user interface, and MariaDB for database management (MariaDB Foundation).

Flask was used to develop the backend because it provides a lightweight framework for creating web applications and handling requests between the user interface and the database. Different routes were created for the system's modules. These routes handle operations such as displaying records, adding new records, updating existing records, deleting records, and searching for information.

The frontend was implemented using HTML templates and CSS. The templates provide the page structure, while CSS is used to improve the application's appearance and layout. A common base template was used to maintain a consistent navigation bar, page structure, and general appearance across the system.

MariaDB was used as the relational database management system. The database contains tables for the main entities of the supermarket, including products, categories, suppliers, customers, employees, purchases, sales, discounts, loyalty cards, and inventory. The system also uses database operations such as views, stored procedures, functions, and triggers where appropriate. These database features help automate some operations and reduce unnecessary manual processing. For example, when a product is sold, the inventory quantity can be updated automatically.

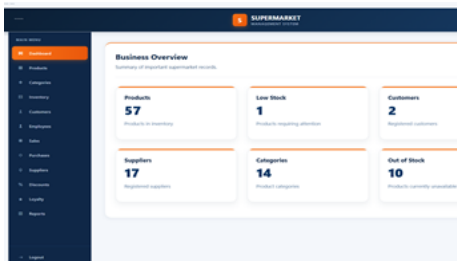
The application was developed and tested locally using a Python virtual environment. This helped keep the required Python packages and project dependencies separate from other projects on the computer.

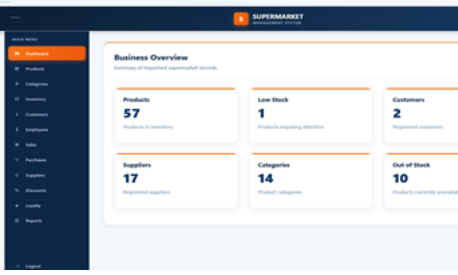
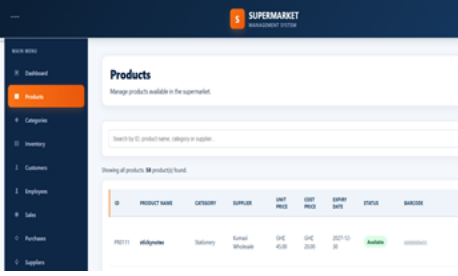
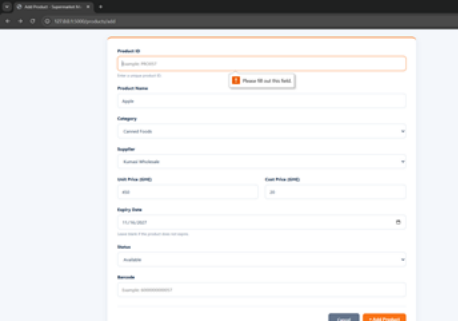
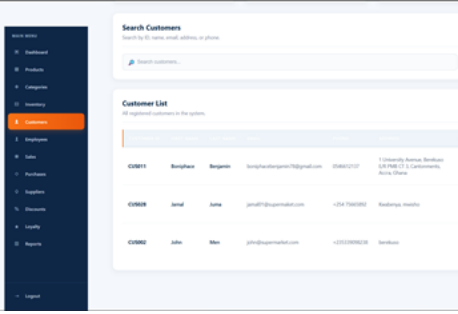
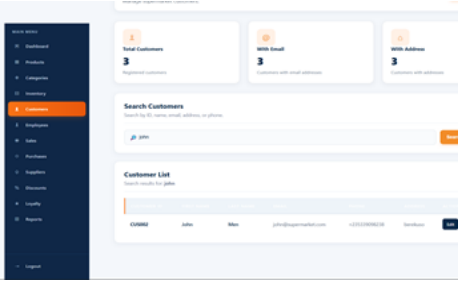
9. Testing Strategy

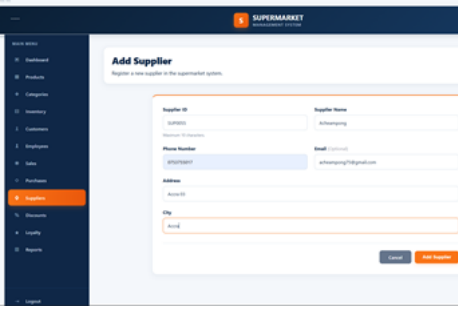
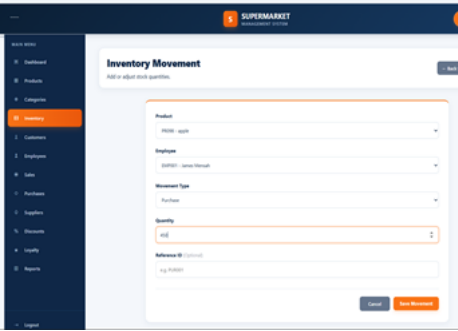
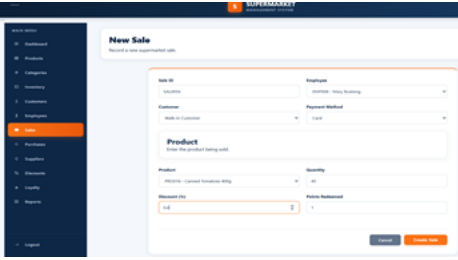
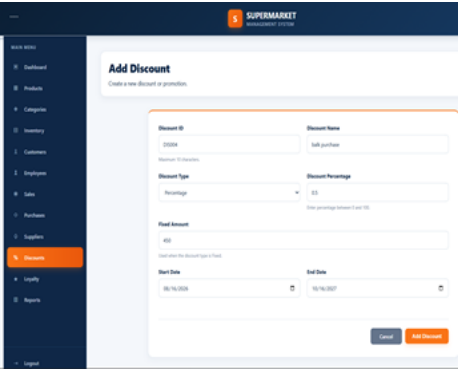
Testing was conducted to determine whether the Supermarket Management System functions perfectly, as per the project requirements. Modules and the complete application were tested to identify errors, verify database operations, and ensure that the expected results were produced.

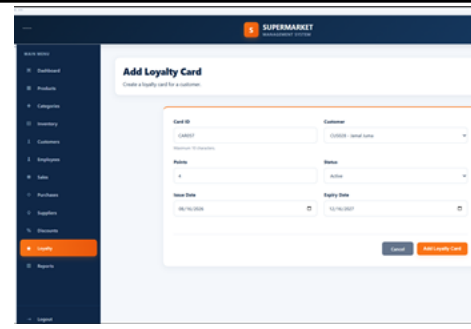
- **Functional Testing:** Each module was tested to ensure its core functions worked correctly. For example, the product management section was tested by adding, editing, viewing, searching, and deleting products. Other sections including categories, suppliers, customers, employees, discounts, loyalty cards, purchases, inventory, and sales were all tested.
- **Database Testing:** The connection between the Flask application and MariaDB was tested to ensure that data could be successfully inserted, retrieved, updated, and deleted. Database relationships were also tested to make sure that records in related tables remain consistent. For example, a sale should contain valid product information and should not reference a product that does not exist.
- **Form Testing:** Forms were tested with various input types to ensure the application handles valid and invalid data correctly. Required fields were checked, and incorrect or incomplete information was tested to ensure that appropriate error messages were displayed.
- **Sales and Inventory Testing:** The sales section was tested to ensure that products can be selected and sales can be recorded correctly. Inventory updates were also checked to confirm that a product's quantity decreases after a successful sale. Purchase operations were similarly tested to confirm that stock quantities increase when new products are received from suppliers.

9.3 Expected and actual result.

TEST	FEATURES	TEST INPUT	EXPECTED RESULTS	ACTUAL RESULT	STATUS
T01	Login (administrator)	Administrator credentials	Admin to login		Pass

T02	Login(cashier)	Cashier Credentials	Cashier to login		Pass
T03	Add Product	Valid Product	To add product		Pass
T04	Add Product	Missing required field	Give an Error or ask to fill the field		Pass
T05	Customer	Add Customer	Add New customer		Pass
T06	Customer	Search Customer	Look for a particular customer		Pass

T07	Supplier	Add Supplier	Add new supplier		Pass
T08	Inventory	Add/ Update Stock	Add Inventory		Pass
T010	Sales	Create valid sale	Add New sales		Pass
T12	Discounts	Discount Applied to sale	Discount to sales		Pass

T13	Loyalty Card	Create Loyalty Card	Add new loyalty card		Pass
-----	--------------	---------------------	----------------------	--	------

10. Security and Data Integrity

10.1 Authentication and authorisation: Security and data integrity were important considerations during the development of the Supermarket Management System. Since the application stores business information, it is vital to the supermarket's day-to-day operations. It is important to protect the information from unauthorised access and accidental modification. Users must first select their job role and enter their username and password before being granted access to the system. The application verifies the provided credentials against the user information stored in the MariaDB database. The password was securely hashed rather than stored in plain text to protect the users' accounts. After the user logs in, the system checks their assigned role and displays the modules and functions they have access to. This helps prevent unauthorised users from accessing or modifying sensitive information.

10.2 Constraints: The system uses database constraints such as primary keys, foreign keys, NOT NULL constraints, and unique constraints to maintain data integrity. These constraints help prevent invalid or incomplete records from being stored in the database. Foreign keys are particularly important because they ensure that relationships between tables remain valid. For example, a sale cannot reference a product that does not exist in the product table. Input validation was also considered when handling information submitted through forms. Users are required to provide appropriate information before records can be successfully submitted.

The application also uses controlled database connections so that communication between the Flask application and MariaDB is handled through the backend rather than allowing users to interact directly with the database. Database transactions and automated database operations help

maintain consistency during important operations. For example, when a sale is completed, the corresponding inventory quantity must be updated correctly.

10.3 Backup consideration: To ensure that data is not lost when the database is corrupted, we will use MariaDB's mariadb-dump utility to create backups of the database. The backup process involves exporting the structure and data of the supermarketdb database into an SQL file. This file will contain the database tables, relationships, and stored data, allowing the database to be recreated if the original database is lost or becomes corrupted. The database will be stored in a secure location separate from the main database, with access restricted to authorized personnel. If the database needs to be restored, the SQL backup file can be imported back into MariaDB, allowing the supermarket's data to be recovered and the system to continue operating.

11. Challenges and Solutions

During the development of the Supermarket Management System, several technical and implementation challenges were encountered.

11.1 Database Connection Problems

One of the challenges was establishing a reliable connection between the Flask application and MariaDB. Incorrect database credentials, connection settings, or configuration issues could prevent the application from accessing the database.

Solution:

The database connection was tested separately before integrating it with the main application. The database host, port, username, password, and database name were verified to match the MariaDB configuration.

11.2 Managing a Large Application

As more modules were added to the system, the Flask application became larger and more difficult to manage. Changes made in one section could affect another section.

Solution:

The application was organized into separate sections based on its functions. Related routes and operations were grouped, making the code easier to identify, maintain, and debug.

11.3 Database Relationship Errors

Another challenge was maintaining correct relationships between different database tables. For example, sales, purchases, products, customers, and suppliers depend on information stored in other tables.

Solution:

Primary keys and foreign keys were used to maintain relationships between tables. The database structure was carefully checked before implementing the corresponding application features.

11.4 Sales and Inventory Updates

The sales module presented a challenge because recording a sale also required correctly updating the inventory quantity. Improper handling could cause stock quantities to become inaccurate.

Solution:

Database logic was used to automate inventory updates after sales and purchases. The sales and inventory functions were also tested using different products and quantities.

11.5 User Interface and Layout Issues

During development, some pages did not initially have the desired layout. Elements could appear mispositioned or fail to be displayed consistently across sections.

Solution:

The HTML templates and CSS files were reviewed and modified to provide a consistent layout. A common base template was also used to maintain similar navigation and page structure throughout the application.

11.6 Debugging Errors

Various errors occurred during the integration of the frontend, Flask backend, and MariaDB database. Identifying the exact source of an error was sometimes difficult because an issue in one component could affect another component.

Solution:

The system was tested one section at a time. Error messages, database responses, and application behaviour were examined to identify the source of each problem. After making corrections, the affected functionality was tested again.

11.7 Maintaining Data Accuracy

Ensuring that information entered into the system remained accurate and consistent was another challenge.

Solution:

Database constraints, validation, relationships, and automated database operations were used to reduce the possibility of invalid or inconsistent data.

Conclusion

Despite these challenges, the development process provided practical experience in database management, backend development, frontend design, system integration, testing, and debugging. The final system provides a centralised platform for managing important supermarket operations and demonstrates how a computerised information system can improve the efficiency and organization of supermarket activities.

APPENDICES

• Database/schema

Figure 1.1 Creation and Selection of the SupermarketDB Database

Shows the creation of the SupermarketDB database and the selection of the database for subsequent SQL operations.

```
MariaDB [(none)]> CREATE DATABASE SupermarketDB;  
Query OK, 1 row affected (0.004 sec)  
  
MariaDB [(none)]> USE SupermarketDB;  
Database changed
```

• Tables

Figure 1.2. SupermarketDB Table Listing

Shows the tables implemented in the SupermarketDB database, including Customer, Employee, Product, Inventory, Purchase, Sale, and their related tables and views

```
MariaDB [SupermarketDB]> SHOW TABLES;  
+-----+  
| Tables_in_supermarketdb |  
+-----+  
| activediscounts  
| category  
| customer  
| customerpurchasehistory  
| discount  
| employee  
| inventory  
| inventorymovement  
| loyaltycard  
| product  
| productdiscount  
| productspercategory  
| productstockstatus  
| purchase  
| purchaseitem  
| sale  
| saleitem  
| salessummary  
| supplier  
| supplierpurchasesummary  
+-----+  
20 rows in set (0.067 sec)
```


- **Table structures: showing primary keys, foreign keys**

Figures 1. 3. Product & Sale Table Structure

Shows the structure of the Product & Sale table, including its attributes, data types, primary key, foreign keys, default values, and unique constraints.

Sample:

```
MariaDB [SupermarketDB]> DESCRIBE Product;
```

Field	Type	Null	Key	Default	Extra
productID	varchar(10)	NO	PRI	NULL	
categoryID	varchar(10)	NO	MUL	NULL	
supplierID	varchar(10)	NO	MUL	NULL	
productName	varchar(100)	NO	UNI	NULL	
unitPrice	decimal(10,2)	NO		NULL	
costPrice	decimal(10,2)	NO		NULL	
expiryDate	date	YES		NULL	
status	enum('Available','Out of Stock','Discontinued')	YES		Available	
barcode	varchar(50)	YES	UNI	NULL	


```
MariaDB [SupermarketDB]> DESCRIBE Sale;
```

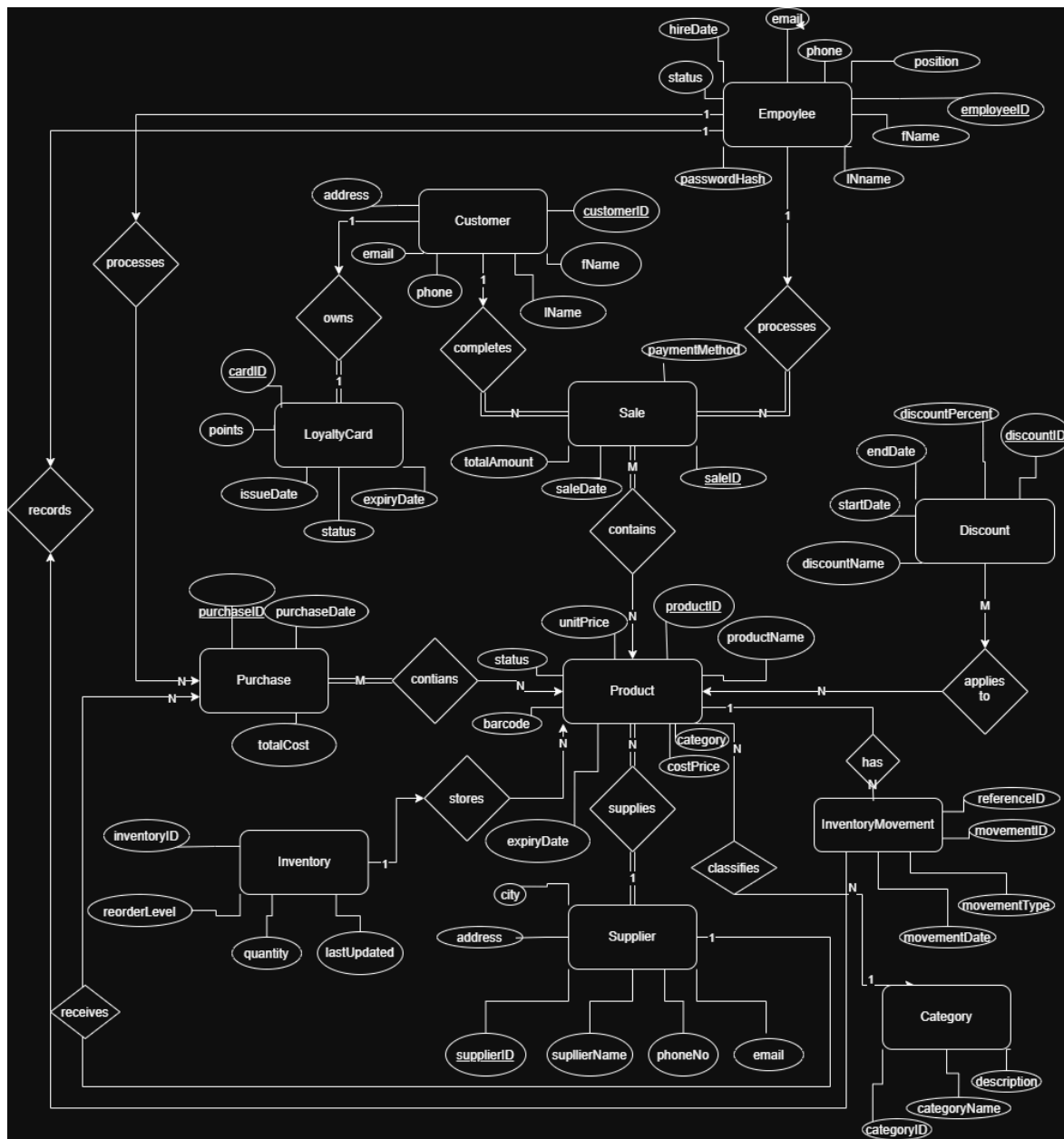
Field	Type	Null	Key	Default	Extra
saleID	varchar(10)	NO	PRI	NULL	
customerID	varchar(10)	YES	MUL	NULL	
employeeID	varchar(10)	NO	MUL	NULL	
saleDate	datetime	YES		current_timestamp()	
totalAmount	decimal(12,2)	YES		0.00	
paymentMethod	enum('Cash','Card','Mobile Money')	NO		NULL	
taxAmount	decimal(12,2)	NO		0.00	
pointsRedeemed	int(11)	NO		0	

8 rows in set (0.061 sec)

- **Relationships**

Figure 1.4 SupermarketDB Entity-Relationship Diagram

Shows the entities, attributes, primary and foreign key relationships, and cardinalities implemented in the SupermarketDB database.



• Sample records

Figure 1.5. Sample Supplier Records

Shows sample supplier records inserted into the SupermarketDB database.

```
MySQL Client (MariaDB 12.2 (x64)) - "C:\Program Files\MariaDB 12.2\bin\mysql.exe" --defaults-file=C:\Program Files\MariaDB 12.2\data\my.ini -u root -p

MariaDB [SupermarketDB]> INSERT INTO Supplier (supplierID, supplierName, phoneNo, email, address, city) VALUES
-> ('SUP001', 'Accra Foods Ltd', '+233240000001', 'info@accrafoods.com', '12 Ring Road', 'Accra'),
-> ('SUP002', 'Golden Harvest Ltd', '+233240000002', 'sales@goldenharvest.com', '25 Spintex Road', 'Accra'),
-> ('SUP003', 'Fresh Choice Ghana', '+233240000003', 'info@freshchoice.com', '8 Liberation Road', 'Accra'),
-> ('SUP004', 'Kumasi Wholesale', '+233240000004', 'sales@kumasiholesale.com', '15 Adum Road', 'Kumasi'),
-> ('SUP005', 'Prime Beverages', '+233240000005', 'info@primebeverages.com', '22 Industrial Area', 'Tema'),
-> ('SUP006', 'Healthy Dairy Ltd', '+233240000006', 'sales@healthydairy.com', '17 Lake Road', 'Accra'),
-> ('SUP007', 'Best Bakery Supply', '+233240000007', 'info@bestbakery.com', '9 Market Street', 'Kumasi'),
-> ('SUP008', 'Royal Canned Foods', '+233240000008', 'info@royalcanned.com', '11 Factory Road', 'Accra'),
-> ('SUP009', 'Snack World Ltd', '+233240000009', 'sales@snackworld.com', '18 North Industrial Area', 'Accra'),
-> ('SUP010', 'Morning Foods Ltd', '+233240000010', 'info@morningfoods.com', '7 Airport Road', 'Accra'),
-> ('SUP011', 'Golden Grain Ltd', '+233240000011', 'sales@goldengrain.com', '14 Industrial Road', 'Tema'),
-> ('SUP012', 'Pure Oils Ghana', '+233240000012', 'sales@pureoils.com', '10 Oil Road', 'Tema'),
-> ('SUP013', 'Clean Home Ltd', '+233240000013', 'info@cleanhome.com', '20 Household Road', 'Accra'),
-> ('SUP014', 'Stationery Hub Ltd', '+233240000014', 'sales@stationeryhub.com', '13 School Road', 'Accra'),
-> ('SUP015', 'Sweet Treats Ltd', '+233240000015', 'sales@sweettreats.com', '2 Chocolate Road', 'Accra'),
-> ('SUP016', 'Pure Water Ghana', '+233240000016', 'info@purewaterghana.com', '30 Water Road', 'Tema');
Query OK, 16 rows affected (0.029 sec)
Records: 16 Duplicates: 0 Warnings: 0

MariaDB [SupermarketDB]> INSERT INTO Customer (customerID, fName, lName, email, address, phone) VALUES
-> ('CUS001', 'John', 'Mensah', 'john.mensah@gmail.com', 'East Legon', '+233500000001'),
-> ('CUS002', 'Ama', 'Boateng', 'ama.boateng@gmail.com', 'Osu', '+233500000002'),
-> ('CUS003', 'Kwame', 'Asante', 'kwame.asante@gmail.com', 'Kumasi', '+233500000003'),
-> ('CUS004', 'Akosua', 'Owusu', 'akosua.owusu@gmail.com', 'Madina', '+233500000004'),
-> ('CUS005', 'Daniel', 'Kofi', 'daniel.kofi@gmail.com', 'Tema', '+233500000005'),
-> ('CUS006', 'Abena', 'Osei', 'abena.osei@gmail.com', 'Adenta', '+233500000006'),
-> ('CUS007', 'Michael', 'Addo', 'michael.addo@gmail.com', 'Spintex', '+233500000007'),
-> ('CUS008', 'Linda', 'Amoah', 'linda.amoah@gmail.com', 'Dansoman', '+233500000008'),
-> ('CUS009', 'Emmanuel', 'Darko', 'emmanuel.darko@gmail.com', 'Kaneshie', '+233500000009'),
-> ('CUS010', 'Grace', 'Yeboah', 'grace.yeboah@gmail.com', 'Airport', '+233500000010'),
-> ('CUS011', 'Samuel', 'Appiah', 'samuel.appiah@gmail.com', 'Achimota', '+233500000011'),
-> ('CUS012', 'Mary', 'Frimpong', 'mary.frimpong@gmail.com', 'Lapaz', '+233500000012'),
-> ('CUS013', 'David', 'Antwi', 'david.antwi@gmail.com', 'Teshie', '+233500000013'),
-> ('CUS014', 'Esther', 'Badu', 'esther.badu@gmail.com', 'Nungua', '+233500000014'),
-> ('CUS015', 'Joseph', 'Adjei', 'joseph.adjei@gmail.com', 'Labadi', '+233500000015'),
-> ('CUS016', 'Patricia', 'Agyeman', 'patricia.agyeman@gmail.com', 'Cantonments', '+233500000016'),
```

Figure 1.6. Sample Customer Records * 10 Sale Records

Shows sample customer records inserted into the SupermarketDB database and displays 10 sale records

```
MySQL Client (MariaDB 12.2 (x64)) - "C:\Program Files\MariaDB 12.2\bin\mysql.exe" --defaults-file=C:\Program Files\MariaDB 12.2\data\my.ini -u root -p

MariaDB [SupermarketDB]> INSERT INTO Supplier (supplierID, supplierName, phoneNo, email, address, city) VALUES
-> ('SUP001', 'Accra Foods Ltd', '+233240000001', 'info@accrafoods.com', '12 Ring Road', 'Accra'),
-> ('SUP002', 'Golden Harvest Ltd', '+233240000002', 'sales@goldenharvest.com', '25 Spintex Road', 'Accra'),
-> ('SUP003', 'Fresh Choice Ghana', '+233240000003', 'info@freshchoice.com', '8 Liberation Road', 'Accra'),
-> ('SUP004', 'Kumasi Wholesale', '+233240000004', 'sales@kumasiholesale.com', '15 Adum Road', 'Kumasi'),
-> ('SUP005', 'Prime Beverages', '+233240000005', 'info@primebeverages.com', '22 Industrial Area', 'Tema'),
-> ('SUP006', 'Healthy Dairy Ltd', '+233240000006', 'sales@healthydairy.com', '17 Lake Road', 'Accra'),
-> ('SUP007', 'Best Bakery Supply', '+233240000007', 'info@bestbakery.com', '9 Market Street', 'Kumasi'),
-> ('SUP008', 'Royal Canned Foods', '+233240000008', 'info@royalcanned.com', '11 Factory Road', 'Accra'),
-> ('SUP009', 'Snack World Ltd', '+233240000009', 'sales@snackworld.com', '18 North Industrial Area', 'Accra'),
-> ('SUP010', 'Morning Foods Ltd', '+233240000010', 'info@morningfoods.com', '7 Airport Road', 'Accra'),
-> ('SUP011', 'Golden Grain Ltd', '+233240000011', 'sales@goldengrain.com', '14 Industrial Road', 'Tema'),
-> ('SUP012', 'Pure Oils Ghana', '+233240000012', 'sales@pureoils.com', '10 Oil Road', 'Tema'),
-> ('SUP013', 'Clean Home Ltd', '+233240000013', 'info@cleanhome.com', '20 Household Road', 'Accra'),
-> ('SUP014', 'Stationery Hub Ltd', '+233240000014', 'sales@stationeryhub.com', '13 School Road', 'Accra'),
-> ('SUP015', 'Sweet Treats Ltd', '+233240000015', 'sales@sweettreats.com', '2 Chocolate Road', 'Accra'),
-> ('SUP016', 'Pure Water Ghana', '+233240000016', 'info@purewaterghana.com', '30 Water Road', 'Tema');
Query OK, 16 rows affected (0.029 sec)
Records: 16 Duplicates: 0 Warnings: 0

MariaDB [SupermarketDB]> INSERT INTO Customer (customerID, fName, lName, email, address, phone) VALUES
-> ('CUS001', 'John', 'Mensah', 'john.mensah@gmail.com', 'East Legon', '+233500000001'),
-> ('CUS002', 'Ama', 'Boateng', 'ama.boateng@gmail.com', 'Osu', '+233500000002'),
-> ('CUS003', 'Kwame', 'Asante', 'kwame.asante@gmail.com', 'Kumasi', '+233500000003'),
-> ('CUS004', 'Akosua', 'Owusu', 'akosua.owusu@gmail.com', 'Madina', '+233500000004'),
-> ('CUS005', 'Daniel', 'Kofi', 'daniel.kofi@gmail.com', 'Tema', '+233500000005'),
-> ('CUS006', 'Abena', 'Osei', 'abena.osei@gmail.com', 'Adenta', '+233500000006'),
-> ('CUS007', 'Michael', 'Addo', 'michael.addo@gmail.com', 'Spintex', '+233500000007'),
-> ('CUS008', 'Linda', 'Amoah', 'linda.amoah@gmail.com', 'Dansoman', '+233500000008'),
-> ('CUS009', 'Emmanuel', 'Darko', 'emmanuel.darko@gmail.com', 'Kaneshie', '+233500000009'),
-> ('CUS010', 'Grace', 'Yeboah', 'grace.yeboah@gmail.com', 'Airport', '+233500000010'),
-> ('CUS011', 'Samuel', 'Appiah', 'samuel.appiah@gmail.com', 'Achimota', '+233500000011'),
-> ('CUS012', 'Mary', 'Frimpong', 'mary.frimpong@gmail.com', 'Lapaz', '+233500000012'),
-> ('CUS013', 'David', 'Antwi', 'david.antwi@gmail.com', 'Teshie', '+233500000013'),
-> ('CUS014', 'Esther', 'Badu', 'esther.badu@gmail.com', 'Nungua', '+233500000014'),
-> ('CUS015', 'Joseph', 'Adjei', 'joseph.adjei@gmail.com', 'Labadi', '+233500000015'),
-> ('CUS016', 'Patricia', 'Agyeman', 'patricia.agyeman@gmail.com', 'Cantonments', '+233500000016'),
```

```
MariaDB [SupermarketDB]> SELECT * FROM Sale LIMIT 10;
```

saleID	customerID	employeeID	saleDate	totalAmount	paymentMethod	taxAmount	pointsRedeemed
SAL001	CUS001	EMP001	2026-08-01 00:00:00	528.65	Cash	15.40	10
SAL002	CUS002	EMP004	2026-08-01 00:00:00	23.48	Card	0.68	0
SAL003	CUS003	EMP007	2026-08-01 00:00:00	122.36	Mobile Money	3.56	0
SAL004	CUS004	EMP002	2026-08-01 00:00:00	282.22	Cash	8.22	20
SAL005	CUS005	EMP005	2026-08-02 00:00:00	49.90	Card	1.45	0
SAL006	CUS006	EMP008	2026-08-02 00:00:00	254.72	Mobile Money	7.42	0
SAL007	CUS007	EMP003	2026-08-02 00:00:00	280.16	Cash	8.16	0
SAL008	CUS008	EMP006	2026-08-02 00:00:00	18.39	Card	0.54	0
SAL009	CUS009	EMP001	2026-08-03 00:00:00	181.28	Mobile Money	5.28	10
SAL010	CUS010	EMP004	2026-08-03 00:00:00	148.32	Cash	4.32	10

10 rows in set (0.054 sec)

• Important SQL queries & results

Advanced queries:

Figure 1.7. Employee Sales Performance Query and Results

Shows an SQL query that calculates the number of transactions and total revenue processed by each employee, filtering employees based on total revenue and ordering the results by revenue.

```
MySQL Client (MariaDB 12.2 (x64)) - "C:\Program Files\MariaDB 12.2\bin\mysql.exe" "--defaults-file=C:\Program Files\MariaDB 12.2\data\my.ini" -uroot -p
```

56 rows in set (0.003 sec)

```
MariaDB [SupermarketDB]> SELECT
->   e.employeeID,
->   CONCAT(e.fName, ' ', e.lName) AS cashierName,
->   e.position,
->   COUNT(s.saleID) AS totalTransactions,
->   SUM(s.totalAmount) AS totalRevenueProcessed
-> FROM Employee e
-> JOIN Sale s
->   ON e.employeeID = s.employeeID
-> GROUP BY
->   e.employeeID,
->   e.fName,
->   e.lName,
->   e.position
-> HAVING totalRevenueProcessed >= 100.00
-> ORDER BY totalRevenueProcessed DESC;
```

employeeID	cashierName	position	totalTransactions	totalRevenueProcessed
EMP002	Sarah Owusu	Cashier	4	997.86
EMP001	James Mensah	Manager	4	970.31
EMP004	Linda Adjei	Supervisor	4	827.14
EMP007	David Osei	Accountant	4	767.55
EMP008	Mary Boateng	Cashier	4	683.71
EMP003	Peter Asante	Cashier	3	490.59
EMP005	Michael Kofi	Stock Clerk	4	347.78
EMP006	Grace Amoah	Cashier	3	325.33

8 rows in set (0.027 sec)

```
MariaDB [SupermarketDB]> SELECT
->   cat.categoryID,
->   cat.categoryName,
->   SUM(si.quantity) AS totalUnitsSold,
->   SUM(si.quantity * si.unitPrice) AS grossRevenue,
```

Figure 1. 8. Customer Purchase and Loyalty Analysis Query and Results

Shows an SQL query that combines customer, loyalty card, and sales data to calculate loyalty points, total orders, and total spending for customers.

```

MySQL Client (MariaDB 12.2 (x64)) - "C:\Program Files\MariaDB 12.2\bin\mysql.exe" --defaults-file=C:\Program Files\MariaDB 12.2\data\my.ini --uroot -p
MariaDB [SupermarketDB]> SELECT
-> c.customerID,
-> CONCAT(c.fName, ' ', c.lName) AS customerName,
-> c.phone,
-> lc.points AS loyaltyPoints,
-> COUNT(s.saleID) AS totalOrders,
-> SUM(s.totalAmount) AS totalSpent
-> FROM Customer c
-> JOIN LoyaltyCard lc
-> ON c.customerID = lc.customerID
-> JOIN Sale s
-> ON c.customerID = s.customerID
-> GROUP BY
-> c.customerID,
-> c.fName,
-> c.lName,
-> c.phone,
-> lc.points
-> ORDER BY totalSpent DESC
-> LIMIT 5;
+-----+-----+-----+-----+-----+
| customerID | customerName | phone | loyaltyPoints | totalOrders | totalSpent |
+-----+-----+-----+-----+-----+
| CUS001 | John Mensah | +233500000001 | 90 | 2 | 924.43 |
| CUS004 | Akosua Owusu | +233500000004 | 145 | 2 | 475.04 |
| CUS002 | Ama Boateng | +233500000002 | 180 | 2 | 429.30 |
| CUS003 | Kwame Asante | +233500000003 | 300 | 2 | 404.37 |
| CUS020 | Sarah Aidoo | +233500000020 | 300 | 1 | 289.43 |
+-----+-----+-----+-----+-----+
5 rows in set (0.046 sec)

```

Figure 1.9. SQL Statements for Creating Database Views

Shows the SQL statements used to create the database views, including CustomerPurchaseHistory, ActiveDiscounts, SupplierPurchaseSummary, and SalesSummary.

```

MySQL Client (MariaDB 12.2 (x64)) - "C:\Program Files\MariaDB 12.2\bin\mysql.exe" --defaults-file=C:\Program Files\MariaDB 12.2\data\my.ini --uroot -p
MariaDB [SupermarketDB]> CREATE VIEW CustomerPurchaseHistory AS
-> SELECT c.customerID, CONCAT(c.fName, ' ', c.lName) AS customerName, s.saleID, s.saleDate,
-> p.productName, si.quantity, si.unitPrice, si.discount, si.subTotal
-> FROM Customer c
-> JOIN Sale s ON c.customerID = s.customerID
-> JOIN SaleItem si ON s.saleID = si.saleID
-> JOIN Product p ON si.productID = p.productID;
Query OK, 0 rows affected (0.014 sec)

MariaDB [SupermarketDB]> CREATE VIEW ActiveDiscounts AS
-> SELECT d.discountID, d.discountName, d.discountType, d.discountPercent, d.fixedAmount,
-> d.startDate, d.endDate, p.productID, p.productName
-> FROM Discount d
-> JOIN ProductDiscount pd ON d.discountID = pd.discountID
-> JOIN Product p ON pd.productID = p.productID
-> WHERE CURDATE() BETWEEN d.startDate AND d.endDate;
Query OK, 0 rows affected (0.042 sec)

MariaDB [SupermarketDB]>
MariaDB [SupermarketDB]> CREATE VIEW SupplierPurchaseSummary AS
-> SELECT s.supplierID, s.supplierName, COUNT(DISTINCT pu.purchaseID) AS totalPurchases,
-> COALESCE(SUM(pu.totalCost), 0) AS totalPurchaseCost
-> FROM Supplier s
-> LEFT JOIN Purchase pu ON s.supplierID = pu.supplierID
-> GROUP BY s.supplierID, s.supplierName;
Query OK, 0 rows affected (0.036 sec)

MariaDB [SupermarketDB]> CREATE VIEW SalesSummary AS
-> SELECT DATE(s.saleDate) AS saleDate, COUNT(DISTINCT s.saleID) AS totalSales,
-> COALESCE(SUM(si.subTotal), 0) AS totalRevenue,
-> COALESCE(SUM(s.pointsRedeemed), 0) AS totalPointsRedeemed
-> FROM Sale s
-> LEFT JOIN SaleItem si ON s.saleID = si.saleID
-> GROUP BY DATE(s.saleDate);
Query OK, 0 rows affected (0.038 sec)

MariaDB [SupermarketDB]> CREATE VIEW ProductsPerCategory AS

```

Views:

Figure 1.10. Customer Purchase History View

Shows the results generated by the CustomerPurchaseHistory view, providing customer, sale, product, quantity, pricing, discount, and subtotal information.

```
MariaDB [supermarketdb]> SELECT * FROM CustomerPurchaseHistory;
```

customerID	customerName	saleID	saleDate	productName	quantity	unitPrice	discount	subTotal
CUS001	John Mensah	SAL001	2026-08-01 00:00:00	Oats 500g	2	26.00	0.00	52.0
CUS001	John Mensah	SAL001	2026-08-01 00:00:00	Penne Pasta 500g	5	17.00	10.00	76.5
CUS001	John Mensah	SAL001	2026-08-01 00:00:00	A4 Printing Paper (Ream)	9	45.00	5.00	384.7
CUS002	Ama Boateng	SAL002	2026-08-01 00:00:00	Chocolate Bar	3	8.00	5.00	22.8
CUS003	Kwame Asante	SAL003	2026-08-01 00:00:00	Sprite 500ml	9	8.00	10.00	64.8
CUS003	Kwame Asante	SAL003	2026-08-01 00:00:00	Bathing Soap	6	9.00	0.00	54.0
CUS004	Akosua Owusu	SAL004	2026-08-01 00:00:00	Penne Pasta 500g	2	17.00	0.00	34.0
CUS004	Akosua Owusu	SAL004	2026-08-01 00:00:00	Palm Oil 1L	8	30.00	0.00	240.0
CUS005	Daniel Kofi	SAL005	2026-08-02 00:00:00	Bathing Soap	5	9.00	15.00	38.2
CUS005	Daniel Kofi	SAL005	2026-08-02 00:00:00	Notebook	1	12.00	15.00	10.2
CUS006	Abena Osei	SAL006	2026-08-02 00:00:00	Spaghetti 500g	8	16.00	0.00	128.0

Figure 1.11. Active Discounts View

Shows currently active discounts together with their discount details, applicable dates, products, and discount values.

```
MariaDB [supermarketdb]> SELECT * FROM ActiveDiscounts;
```

discountID	discountName	discountType	discountPercent	fixedAmount	startDate	endDate	productID	productName
DIS008	Back To School	Percentage	15.00	0.00	2026-08-01	2026-09-30	PR0008	Butter 250g
DIS011	Loyalty Discount	Percentage	5.00	0.00	2026-01-01	2026-12-31	PR0011	Sugar Bread
DIS012	Bulk Purchase	Percentage	10.00	0.00	2026-01-01	2026-12-31	PR0012	Chocolate Cake

3 rows in set (0.069 sec)

Figure 1.12. Supplier Purchase Summary View

Shows a summary of purchase information for each supplier, including the number of purchases and the total purchase cost.

```
MariaDB [supermarketdb]> SELECT * FROM SupplierPurchaseSummary;
```

supplierID	supplierName	totalPurchases	totalPurchaseCost
SUP001	Accra Foods Ltd	1	2698.00
SUP002	Golden Harvest Ltd	1	1954.00
SUP003	Fresh Choice Ghana	1	3346.00
SUP004	Kumasi Wholesale	1	14362.00
SUP005	Prime Beverages	1	2468.20
SUP006	Healthy Dairy Ltd	1	9661.00
SUP007	Best Bakery Supply	1	5470.00
SUP008	Royal Canned Foods	1	8106.50
SUP009	Snack World Ltd	1	1116.00
SUP010	Morning Foods Ltd	1	12185.00
SUP011	Golden Grain Ltd	1	15692.00
SUP012	Pure Oils Ghana	1	15911.00
SUP013	Clean Home Ltd	1	8913.00
SUP014	Stationery Hub Ltd	1	11558.00
SUP015	Sweet Treats Ltd	1	3244.50
SUP016	Pure Water Ghana	1	818.00

16 rows in set (0.098 sec)

Figure 1.13. Sales Summary View

Shows daily sales information, including total sales, total revenue, and loyalty points redeemed.

```
MariaDB [supermarketdb]> SELECT * FROM SalesSummary;
```

saleDate	totalSales	totalRevenue	totalPointsRedeemed
2026-08-01	4	928.85	70
2026-08-02	4	585.60	0
2026-08-03	4	514.00	50
2026-08-04	4	565.00	70
2026-08-05	4	737.40	20
2026-08-06	4	386.10	40
2026-08-07	4	1278.85	60
2026-08-08	2	256.90	0

8 rows in set (0.026 sec)

• Stored procedures/functions

Procedures:

Figure 1.14. RecordPurchase Stored Procedure

Shows the creation of the RecordPurchase stored procedure for recording purchase transactions and their associated purchase items.

```

MySQL Client (MariaDB 12.2 (x64)) - "C:\Program Files\MariaDB 12.2\bin\mysql.exe" --defaults-file=C:\Program Files\MariaDB 12.2\data\my.ini" -uroot -p
MariaDB [SupermarketDB]>
MariaDB [SupermarketDB]> DELIMITER ;
MariaDB [SupermarketDB]> DELIMITER //
MariaDB [SupermarketDB]> CREATE PROCEDURE RecordPurchase(
-> IN p_purchaseID VARCHAR(10), IN p_supplierID VARCHAR(10), IN p_employeeID VARCHAR(10),
-> IN p_productID VARCHAR(10), IN p_quantity INT, IN p_unitCost DECIMAL(10,2))
-> BEGIN
-> DECLARE v_total DECIMAL(12,2);
-> INSERT INTO Purchase(purchaseID,supplierID,employeeID,purchaseDate,totalCost,status)
-> VALUES(p_purchaseID,p_supplierID,p_employeeID,CURDATE(),p_quantity*p_unitCost,'Completed');
-> INSERT INTO PurchaseItem(purchaseID,productID,quantity,unitCost) VALUES(p_purchaseID,p_productID,p_quantity,p_unitCost);
-> SELECT SUM(quantity*unitCost) INTO v_total FROM PurchaseItem WHERE purchaseID=p_purchaseID;
-> UPDATE Purchase SET totalCost=v_total WHERE purchaseID=p_purchaseID;
-> END //
Query OK, 0 rows affected (0.030 sec)

```

Figure 1.15. SearchProducts Stored Procedure

Shows the creation of the SearchProducts stored procedure for searching products by name, barcode, or category.

```

MariaDB [SupermarketDB]> SHOW CREATE PROCEDURE SearchProducts;
+-----+-----+-----+-----+
| Procedure | sql_mode | Create Procedure |
+-----+-----+-----+-----+
| SearchProducts | STRICT_TRANS_TABLES,ERROR_FOR_DIVISION_BY_ZERO,NO_AUTO_CREATE_USER,NO_ENGINE_SUBSTITUTION | CREATE DEFINER='root'@'localhost' PROCEDURE `SearchProducts`(IN p_search VARCHAR(100))
BEGIN
SELECT p.productID, p.productName, p.barcode, c.categoryName, p.unitPrice, i.quantity, p.status
FROM Product p
JOIN Category c ON p.categoryID = c.categoryID
JOIN Inventory i ON p.productID = i.productID
WHERE p.productName LIKE CONCAT('%', p_search, '%')
OR p.barcode = p_search
OR c.categoryName LIKE CONCAT('%', p_search, '%');
END | utf8mb4 | utf8mb4_uca1400_ai_ci | utf8mb4_uca1400_ai_ci |
+-----+-----+-----+-----+
1 row in set (0.100 sec)

```

Figure 1.16 CalculateSaleTotal Function

Shows the creation of the CalculateSaleTotal function used to calculate the total amount of a sale.


```

MariaDB [SupermarketDB]> SHOW CREATE FUNCTION CalculateSaleTotal;
+-----+-----+-----+-----+
| Function | sql_mode | Create Function |
+-----+-----+-----+-----+
| CalculateSaleTotal | STRICT_TRANS_TABLES,ERROR_FOR_DIVISION_BY_ZERO,NO_AUTO_CREATE_USER,NO_ENGINE_SUBSTITUTION |
+-----+-----+-----+-----+
| character_set_client | collation_connection | Database Collation |
+-----+-----+-----+-----+
| utf8mb4 | utf8mb4_uca1400_ai_ci | utf8mb4_uca1400_ai_ci |
+-----+-----+-----+-----+
1 row in set (0.027 sec)

```

• Trigger

Figure 1.17. AfterSaleItemInsert Trigger

Shows the AfterSaleItemInsert trigger, which automatically updates inventory quantity and the last-updated timestamp when a sale item is inserted.

```

DELIMITER ' at line 1
->
-> CREATE TRIGGER AfterSaleItemInsert
-> AFTER INSERT ON SaleItem
-> FOR EACH ROW
-> BEGIN
->     UPDATE Inventory SET quantity = quantity - NEW.quantity, lastUpdated = CURRENT_TIMESTAMP
->     WHERE productID = NEW.productID;
MariaDB [supermarketdb]> END //
->
-> DELIMITER ;

```

References

Connolly, T., & Begg, C. (2015). *Database systems: A practical approach to design, implementation, and management* (6th ed.). Pearson. MariaDB Foundation. (n.d.). *MariaDB documentation*. MariaDB.org. <https://mariadb.org/documentation/>

Pallets Projects. (n.d.). *Flask documentation*. <https://flask.palletsprojects.com/en/stable/>

Yussif, S., Kusi, A., & Ismail, A. (2019). Record and information management among SMEs in Ghana: Evidence from selected SMEs in the Wa Municipality of Ghana. *International Journal of Academic Research in Business and Social Sciences*, 9(8), 184–199. <https://dx.doi.org/10.6007/IJARBS/v9-i8/6234>